

Project Report: One Prompt Multiple Minds

SECTION 1 — PROJECT SUMMARY

Project Title: One Prompt Multiple Minds (Multi-AI Prompt Response Aggregator)

Aim of the Project: To build a centralized, intelligent platform that allows users to query multiple Large Language Models (LLMs) simultaneously using a single prompt, evaluate their diverse responses, and generate a synthesized "Collective Insight."

Problem Statement: In the rapidly evolving AI landscape, users often need to consult different AI models (such as Gemini, Groq, and Mistral) to obtain the most accurate or well-rounded answer. Manually switching between different AI interfaces, copying prompts, and comparing responses is tedious, time-consuming, and inefficient.

Objective: To develop a responsive web application featuring a React-based frontend and a Flask backend that parallelizes API requests to multiple selected AI models, stores interaction histories in a local database, and utilizes a meta-prompting technique to synthesize a unified response.

Introduction: "One Prompt Multiple Minds" is an innovative web application designed to act as a neural synthesizer. By combining the cognitive capabilities of multiple AI engines, the system provides users with side-by-side comparative analysis of AI outputs, including metrics like word count and response time.

Features:

- **Single Prompt Interface:** Query multiple AI engines concurrently.
- **Concurrent Processing:** Multithreaded backend architecture to minimize wait times.
- **Dynamic AI Selection:** Users can toggle which AI models (Groq, Gemini, Mistral) to query.
- **Meta-Summary Generation:** Automatically generates a "Collective Insight" summarizing the best points from all models.
- **Interaction History:** Persistent storage using SQLite to review or delete past prompts.
- **Elite UI/UX:** Features dynamic simulated streaming text, glassmorphism design, and smooth Framer Motion animations.

Technologies Used:

- **Frontend:** React 19, Vite, React Router DOM, Framer Motion, Axios, Lucide React.
- **Backend:** Python 3, Flask, Flask-CORS, Multithreading.
- **Database:** SQLite3.
- **AI/APIs:** Google Gemini API (`google-generativeai`), Groq API, Mistral API.

Architecture Overview: The system follows a classic Client-Server Architecture. The React frontend captures the user's prompt and selected models, sending a REST API request to the Flask backend. The backend utilizes multithreading to concurrently dispatch requests to the respective AI APIs. Once all responses are gathered, they are fed into a meta-prompt sent to Gemini 2.5 Flash to synthesize a collective summary. The results are then stored in a SQLite database and returned to the frontend.

Working Principle:

1. The user enters a prompt and selects the desired AI engines on the Dashboard.
2. The React frontend sends a POST request to `/api/process`.
3. The Flask backend spawns independent threads for each selected AI service (e.g., `get_groq_response`, `get_gemini_response`).
4. The backend waits for all threads to complete (`t.join()`).
5. A summary service takes the aggregated texts and asks Gemini to generate a single concise insight.
6. The interaction and individual responses are saved to the database.
7. The synthesized data is returned to the client and displayed using dynamic typing animations.

Advantages: - **Time Efficiency:** Eliminates the need to consult multiple AI websites separately. - **Enhanced Accuracy:** Cross-referencing multiple AI models helps in verifying facts and reducing hallucinations. - **Performance Metrics:** Allows users to compare which model replies fastest and verbosely.

Limitations: - Heavily dependent on external API rate limits and network latency. - API keys must be valid and funded (if applicable) for the system to function.

Future Scope: - Integration of more AI models (like OpenAI GPT-4, Anthropic Claude). - User authentication and personalized history. - Exporting results to PDF/Word documents directly from the UI.

Conclusion: This project successfully demonstrates the integration of multiple AI APIs into a cohesive, user-friendly application. It highlights modern web development practices, asynchronous backend processing, and advanced prompt engineering, making it a highly practical tool for researchers and developers.

SECTION 2 — PROJECT WORKFLOW

- User Input:** The user lands on the Home page and enters a query (e.g., "Explain quantum entanglement") and selects the target AI engines (Groq, Gemini, Mistral).
- Frontend Capture:** The React application captures the input state and triggers an Axios POST request payload to the Flask backend.
- API Routing:** The Flask app receives the payload at the `/api/process` endpoint.
- Concurrent Execution:** The backend initializes a threading mechanism, dispatching parallel requests to `groq_service`, `gemini_service`, and `mistral_service`.
- Data Aggregation:** The backend collects the responses, calculating the response time and word count for each engine.
- Meta-Synthesis:** The aggregated responses are passed to `summary_service.py`, which prompts Gemini 2.5 Flash to generate a "Collective Insight".
- Database Storage:** The primary prompt and summary are inserted into the `interactions` table, while individual AI responses are inserted into the `responses` table with a foreign key relationship.
- UI Rendering:** The frontend receives the comprehensive JSON response and routes the user to the Results page, utilizing a "TypedText" component to simulate an AI typing out the response dynamically.

SECTION 3 — COMPLETE FILE & FOLDER EXPLANATION

```
text ccps project/ █ █ █ █ backend/ █ █ █ █ app.py █ █ █ █ database.py █ █ █ █
requirements.txt █ █ █ █ services/ █ █ █ █ gemini_service.py █ █ █ █
groq_service.py █ █ █ █ mistral_service.py █ █ █ █ summary_service.py █ █ █ █
frontend/ █ █ █ █ package.json █ █ █ █ vite.config.js █ █ █ █ src/ █ █ █ █ App.jsx █ █ █ █
pages/ █ █ █ █ Home.jsx █ █ █ █ Results.jsx █ █ █ █ History.jsx
```

backend/

→ Handles server-side processing, database management, API routing, and AI API communication.

backend/app.py → The main Flask application file. It defines the REST API routes (/api/process, /api/history), handles multithreaded concurrent API calls to the AI models to reduce latency, and manages data insertion into the database.

backend/database.py → Responsible for database initialization and connection management. It creates the interactions and responses SQLite tables to persist the prompt history.

backend/requirements.txt → Lists all Python dependencies required for the backend, such as Flask, google-genai, groq, and mistralai.

backend/services/

→ Contains modularized service functions to interact cleanly with external AI APIs.

backend/services/gemini_service.py → Handles authentication and communication with the Google Gemini API to fetch standard responses.

backend/services/groq_service.py → Manages requests and responses specifically for the Groq API.

backend/services/mistral_service.py → Manages requests and responses specifically for the Mistral API.

backend/services/summary_service.py → The core synthesizer logic. It takes the array of responses from all selected models, formats them into a meta-prompt, and uses the Gemini API to generate a final, synthesized "Collective Insight."

frontend/

→ Contains the user interface client application built with React and Vite.

frontend/package.json → Manages Node.js dependencies, project metadata, and scripts (like npm run dev).

frontend/src/App.jsx → The main React component that establishes the application layout, navigation bar, and defines routing using react-router-dom.

frontend/src/pages/

→ Contains the primary page components of the application.

frontend/src/pages/Home.jsx → The interactive Dashboard where users enter their prompt, toggle AI engines, and initiate the synthesis. Features Framer Motion animations.

frontend/src/pages/Results.jsx → Displays the AI responses. Uses a custom TypedText component to simulate real-time text streaming and shows metrics (response time, word count).

frontend/src/pages/History.jsx → Fetches and displays past interactions from the database, allowing users to review older prompts or delete them from the SQLite database.

SECTION 4 — TEAM CONTRIBUTION (4 MEMBERS)

Member 1: Frontend Design & UI/UX Integration - Designed the overarching visual theme and CSS styling (glassmorphism, gradient text). - Developed the core React components (Home.jsx, Results.jsx, History.jsx). - Implemented smooth transitions and micro-animations using Framer Motion and Lucide React icons.

Member 2: Backend Architecture & API Routing - Set up the Flask backend environment and configured CORS. - Developed the RESTful API endpoints (app.py). - Implemented the Python threading module to allow concurrent execution of AI API requests, significantly reducing overall wait time.

Member 3: AI Integrations & Meta-Prompting - Integrated the official SDKs for Groq, Mistral, and Google Gemini. - Developed the modular service architecture inside backend/services/. - Designed the meta-prompt engineering logic inside summary_service.py to ensure high-quality synthesis of collective insights.

Member 4: Database Management, Testing & Documentation - Designed the relational schema and wrote the SQLite integration in database.py. - Developed the endpoints to fetch and delete historical interactions. - Conducted end-to-end API testing, managed environment variables (.env), and compiled the final project documentation and Viva presentation.

SECTION 5 — TECHNICAL DETAILS

Software Requirements: - **OS:** Windows 10/11, macOS, or Linux. - **Node.js:** v18.0.0 or higher. - **Python:** v3.9 or higher. - **Browser:** Google Chrome, Edge, or any modern web browser.

Hardware Requirements: - **Processor:** Intel i3 / AMD Ryzen 3 or higher. - **RAM:** Minimum 4GB (8GB recommended). - **Storage:** Minimum 500MB of free disk space.

Languages Used: - **Frontend:** HTML5, CSS3, JavaScript (JSX). - **Backend:** Python. - **Database Query:** SQL.

Frameworks & Libraries: - **Frontend:** React.js, Vite, React Router DOM, Framer Motion, Axios. - **Backend:** Flask, Flask-CORS.

APIs Used: - Google Gemini API (gemini-2.5-flash) - Groq API - Mistral API

Input/Output Design: - **Input:** Text-based prompts entered via an HTML <textarea>, alongside boolean toggles for AI model selection. - **Output:** Structured JSON responses from the backend, mapped to React components displaying generated text, temporal metrics (seconds), and length metrics (word counts).

System Requirements: - An active internet connection is strictly required for the backend to communicate with the external AI APIs.

SECTION 6 — VIVA PREPARATION

1-Minute Viva Introduction Speech

"Good morning, respected examiners. We are here to present our project titled 'One Prompt Multiple Minds.' In today's world, developers and researchers often rely on various AI models to cross-check information, as different models have different strengths. Doing this manually is highly inefficient. To solve this, we developed a full-stack web application using React and Flask. Our platform takes a single user prompt and uses multithreading to query multiple AI APIs—such as Gemini, Groq, and Mistral—concurrently. Not only does it display the responses side-by-side with performance metrics, but it also uses a meta-prompting technique to synthesize the best points from all models into a single 'Collective Insight'. Finally, it saves all interactions in a SQLite database for future review. Thank you!"

Complete Project Flow in Simple Language

1. You type a question on the website and select which AIs you want to ask.
2. The website sends this question to our Python server.
3. The Python server acts like a dispatcher. Instead of asking one AI and waiting, it asks all selected AIs at the exact same time (using threads).
4. Once all AIs reply, the server gathers their answers and sends them all to one master AI (Gemini), asking it to read everything and write a short summary.
5. The server saves everything in a database file so you don't lose it.
6. The server sends the summary and individual answers back to the website.
7. The website displays it with a cool typing effect.

20 Viva Questions with Answers

What is the main objective of this project? *Answer:* To allow users to query multiple AI models with a single prompt, compare their responses, and get a synthesized summary automatically.

Which frontend framework did you use and why? *Answer:* We used React.js (with Vite). It allows for building dynamic, single-page applications efficiently using reusable components.

What backend framework is handling the server? *Answer:* We used Flask, a lightweight Python web framework, because it is excellent for quickly building REST APIs and integrates easily with Python-based AI SDKs.

How do the frontend and backend communicate? *Answer:* They communicate via RESTful APIs using the Axios library in React to send HTTP POST and GET requests to the Flask server.

Which database is used in this project? *Answer:* SQLite. It is lightweight, file-based, and perfect for local interaction history storage without requiring a heavy database server setup.

Why did you use multithreading in your backend? *Answer:* AI API requests involve network waiting time (I/O bound). By using multithreading, we can request data from Groq, Gemini, and Mistral concurrently, reducing the total wait time to the time of the slowest model, rather than the sum of all models.

What is the 'Collective Insight' feature? *Answer:* It's a meta-summary. After all individual models respond, the `summary_service.py` sends all those responses back to Gemini and asks it to extract the best points into one final, synthesized answer.

What APIs are integrated into this project? *Answer:* Google Gemini API, Groq API, and Mistral API.

What does the CORS library do in your Flask app? *Answer:* Cross-Origin Resource Sharing (CORS) allows our React frontend (running on one port, like 5173) to securely make requests to our Flask backend (running on port 5001).

How does the system track history? *Answer:* When the `/api/process` finishes fetching data, it executes an INSERT SQL query to save the prompt and summary into the `interactions` table, and the individual answers into the `responses` table.

What is Framer Motion used for? *Answer:* It is a React animation library we used to create smooth fade-ins, sliding transitions, and micro-interactions on the frontend.

What challenges did you face during development? *Answer:* A major challenge was managing API rate limits and ensuring the backend didn't crash if one API failed. We handled this using `try-except` blocks in the threads to gracefully catch errors.

How is the response time metric calculated? *Answer:* Before calling the API, we record the start time using `time.time()`, and after the API returns, we subtract the start time from the current time to get the duration in seconds.

How do you calculate the word count? *Answer:* We simply split the returned response string by spaces `len(response.split())`.

What happens if an API key is missing or invalid? *Answer:* The specific service's `try-except` block catches the authentication error and returns an error message as its response, without breaking the rest of the application. The summary service also has a fallback static response.

Why use Vite instead of Create React App? *Answer:* Vite offers significantly faster hot module replacement (HMR) and optimized build times compared to Create React App.

What is the purpose of .env files? *Answer:* To securely store sensitive information like API keys so they are not hardcoded into the source code or uploaded to GitHub.

Explain the table relationships in your database. *Answer:* It's a one-to-many relationship. One `interaction` (the main prompt) has many `responses` (one from each AI engine). The `responses` table uses `interaction_id` as a foreign key.

What is the advantage of aggregating AI responses? *Answer:* Different AIs are trained on different datasets and have distinct biases. Aggregating them provides a more comprehensive, fact-checked, and balanced perspective.

What is a future improvement you could make? *Answer:* Adding user authentication to keep interaction histories private per user, or adding file upload capabilities so the AIs can analyze documents.